Course Code: CSE 499B
Section: 13
Project Group No: 06
Group Members:
Md.Murad Ul Momin (ID: 2232360642)
Rima Saha (ID: 2132517042)
Sakia Tabassum Ananna (ID: 2221433042)
Ashraful Hossain Riyan (ID: 2232845642)

PRIVATE SHIELD: Weekly Update Report

I. INTRODUCTION

This week's work focused on building the core infrastructure of PRIVATE SHIELD, a dataset-independent federated-learning framework. The current implementation includes a common model interface, reusable client and server components, sample-weighted FedAvg aggregation, evaluation utilities, reproducibility controls, and experiment logging. At this stage, small synthetic datasets were used only to verify that the software pipeline works correctly; the resulting metrics should not be interpreted as research results.

II. IMPLEMENTATION WORK

A. Common Model and Dataset Interface

A configurable MLP was implemented in mlp.py, with fresh models created through factory.py. Feature count, class count, and hidden-layer sizes are configurable. datasets.py validates feature tensors and integer labels without fitting preprocessing. dummy.py creates separate client and evaluation fixtures for testing. The default test uses three clients with 16, 24, and 32 training samples, six features, and three artificial classes.

B. Federated Client, Server, and FedAvg

FederatedClient receives global parameters, trains locally using cross-entropy with Adam or SGD, and returns its model, update, sample count, and training loss. FederatedServer selects clients reproducibly and coordinates communication rounds. FedAvgAggregator combines local models using training-sample proportions. For clients with 100 and 300 samples, the weights are 0.25 and 0.75. Parameter utilities safely copy states and calculate local-minus-global updates. History records observed updates by client and round.

C. Metrics, Reproducibility, and Logging

The configuration utility reads and validates YAML settings. Seed controls cover Python, NumPy, and PyTorch, while metadata records the device, versions, and Git state. Evaluation reports accuracy, balanced accuracy, macro precision, recall and F1, weighted F1, per-class metrics, and a confusion matrix. Evaluation data does not influence training or aggregation. The logger creates unique run directories containing settings, metadata, metrics, aggregation weights, timing, and an inspection checkpoint.

III. INTEGRATION AND CURRENT STATUS

main.py connects the components and requires explicit dummy-test execution. This week's validation recorded 30 passing unit and integration tests. Checks covered weighted averaging, parameter isolation, model shapes, deterministic selection, metric calculations, logging, and repeatable runs. Changing held-out labels did not change the trained global model. The framework currently runs in one local Python process. Real-data experiments and exact checkpoint resume remain unavailable.

NEXT WEEK'S OBJECTIVES

- Inspect CICIoT2023 provenance, features, labels, and victim-device mapping before defining partitions.
- Agree on training, auxiliary-validation, and test separation; finalize the plan for next week.
- Integrate approved real data and establish the FedAvg baseline before adding research methods.
- Keep FedMADE, SecureDyn trust detection, poisoning, and the proposed fusion method deferred.